

UNIVERSIDAD NACIONAL DE INGENIERÍA
FACULTAD DE INGENIERÍA INDUSTRIAL Y DE
SISTEMAS



LENGUAJES DE PROGRAMACIÓN I

SW303-U

GRUPO 7

“Sistema de inventario con SQLite3 en C”

PROFESOR: JUAN LUIS HERENCIA GUERRA

ALUMNOS:

BELTRAN MENDOZA, JEAN PAUL TOMAS	20250156B
MENDOZA OSCANOVA, PERCY SAUL	20250088G
CORONEL TELLO, ALVARO JESUS	20231122I
YATACO HUASASQUICHE, JEFERSON SNYDER	20244565A

Lima, 27 de abril de 2026

2026-I

REQUERIMIENTOS DEL SISTEMA

1.1 Requerimientos funcionales

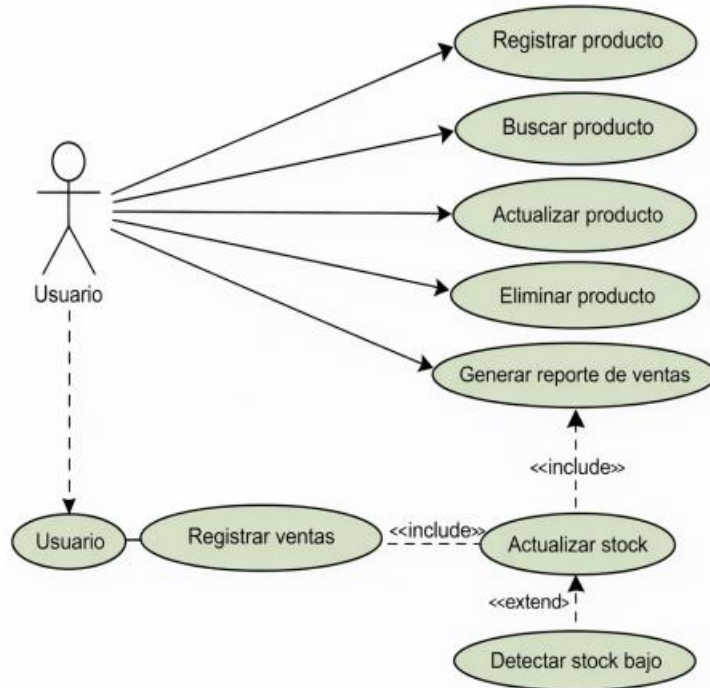
- El sistema debe permitir registrar productos con ID, nombre, categoría, precio y stock.
- El sistema debe permitir actualizar productos existentes.
- El sistema debe permitir eliminar productos.
- El sistema debe permitir listar productos.
- El sistema debe permitir buscar productos por nombre o categoría.
- El sistema debe alertar cuando el stock sea menor a un umbral.
- El sistema debe permitir registrar ventas.
- El sistema debe generar reportes en CSV.
- El sistema debe usar transacciones atómicas.

1.2 Requerimientos no funcionales

- El sistema debe usar SQLite como base de datos.
- El sistema debe desarrollarse en lenguaje C.
- El sistema debe ejecutarse en consola.
- El sistema debe garantizar persistencia de datos.
- El sistema debe ser eficiente.
- El sistema debe ser modular.

MODELADO DE PROCESOS

2.1 Diagrama de casos de uso

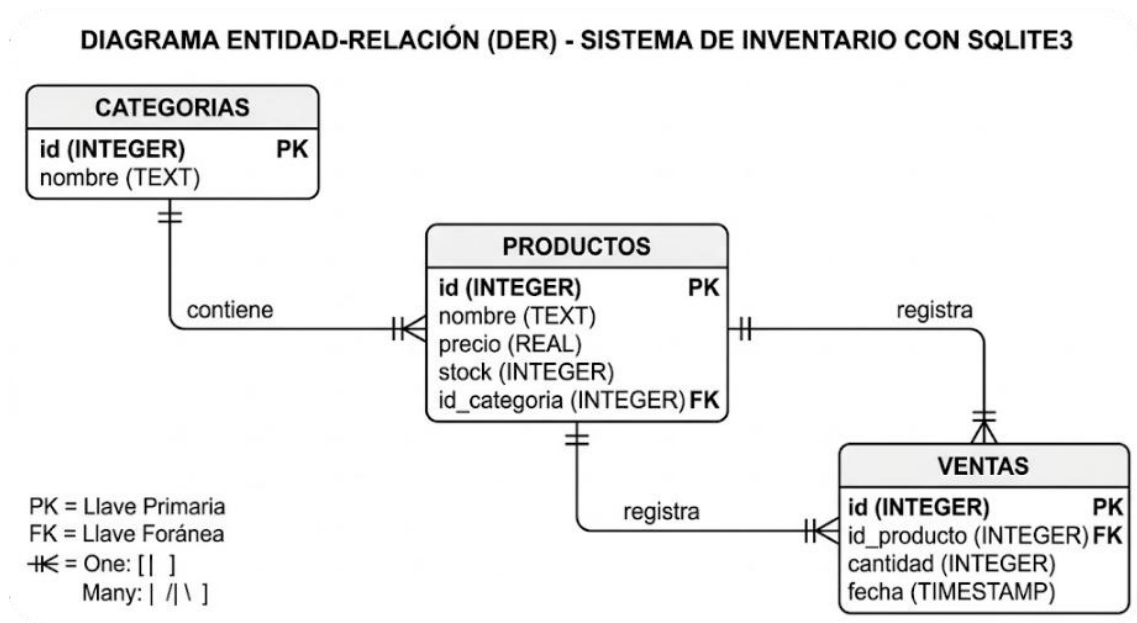


2.2 Descripción de Casos de Uso

- Registrar producto: El usuario ingresa datos y el sistema los guarda.
- Buscar producto: El usuario consulta y el sistema muestra resultados.
- Actualizar producto: El usuario modifica datos existentes.
- Eliminar producto: El usuario elimina un producto.
- Generar reporte: El sistema crea un archivo CSV.
- Detectar stock bajo: El sistema alerta si el stock es bajo

DISEÑO DE LA BASE DE DATOS

3.1 El Diagrama Entidad-Relación (DER)



- **Integridad Referencial:** Las **Llaves Foráneas (FK)** aseguran que no se pueda registrar un producto en una categoría que no existe.
- **Optimización de Tipos:** Se usa **REAL** para el precio (decimales) e **INTEGER** para el stock y cantidades, lo que optimiza el uso de memoria en SQLite3.
- **Escalabilidad:** Al separar las categorías en su propia tabla, el sistema es más ordenado que si solo pusieras el nombre de la categoría como texto dentro de la tabla de productos.

3.2 El Diccionario de Datos

Campo	Tipo de SQLite	Descripción
id	INTEGER	Número único que identifica al producto o venta.

nombre	<i>TEXT</i>	El nombre del producto o la categoría.
precio	<i>REAL</i>	El costo del producto (admite decimales).
stock	<i>INTEGER</i>	Cuántas unidades quedan en el inventario.
fecha	<i>TIMESTAMP</i>	El momento exacto en que se hizo la venta.

3.3 El Script de Creación (SQL)

```

1  -- Crear la tabla de Categorías primero
2  CREATE TABLE categorias (
3      id INTEGER PRIMARY KEY AUTOINCREMENT,
4      nombre TEXT NOT NULL UNIQUE
5  );
6
7  -- Crear la tabla de Productos vinculada a Categorías
8  CREATE TABLE productos (
9      id INTEGER PRIMARY KEY AUTOINCREMENT,
10     nombre TEXT NOT NULL,
11     id_categoria INTEGER,
12     precio REAL NOT NULL,
13     stock INTEGER NOT NULL DEFAULT 0,
14     FOREIGN KEY (id_categoria) REFERENCES categorias(id)
15 );
16
17 -- Crear la tabla de Ventas para los reportes CSV
18 CREATE TABLE ventas (
19     id INTEGER PRIMARY KEY AUTOINCREMENT,
20     id_producto INTEGER NOT NULL,
21     cantidad INTEGER NOT NULL,
22     fecha TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
23     FOREIGN KEY (id_producto) REFERENCES productos(id)
24 );

```

ARQUITECTURA DE SOFTWARE

4.1 Justificación de la Arquitectura de Datos

La arquitectura propuesta para el sistema de inventario se basa en un modelo relacional gestionado por **SQLite3**, estructurado bajo los siguientes pilares de diseño:

A. Normalización y Escalabilidad

Se ha optado por una estructura de tres tablas principales (Categorías, Productos y Ventas) para cumplir con las mejores prácticas de bases de datos:

- **Reducción de Redundancia:** Al separar las categorías en una tabla independiente, evitamos la repetición innecesaria de cadenas de texto en la tabla de productos, optimizando el almacenamiento en disco.
- **Histórico de Movimientos:** La inclusión de una tabla de Ventas no solo permite generar el reporte CSV requerido, sino que garantiza la trazabilidad de cada salida de stock con una marca de tiempo exacta (TIMESTAMP).

B. Garantía de Atomicidad (ACID)

Siguiendo los requerimientos del proyecto sobre **actualizaciones atómicas**, la arquitectura está diseñada para trabajar con transacciones SQL:

- Se justifica el uso de transacciones para operaciones críticas (como registrar una venta), asegurando que la disminución del stock y la creación del registro de venta ocurran simultáneamente. Si una falla, la otra se revierte automáticamente, evitando datos inconsistentes.

C. Patrón DAO (Data Access Object) en C

Para la implementación, se ha diseñado una capa de acceso a datos que utiliza funciones *wrapper* sobre la API `sqlite3_exec`:

- **Encapsulamiento:** El resto del programa (la lógica de inventario) no

necesita conocer los detalles de SQL; solo llama a funciones predefinidas en C.

- **Seguridad:** Al centralizar las consultas, facilitamos el manejo de errores y la limpieza de memoria, evitando fugas de recursos al cerrar la base de datos.

D. Integridad y Reglas de Negocio

Se han implementado restricciones directamente en el motor de la base de datos para delegar la validación de reglas básicas:

- **Restricciones de Dominio:** Uso de cláusulas CHECK para garantizar que el precio y el stock nunca sean valores negativos.
- **Integridad Referencial:** Uso de FOREIGN KEY para asegurar que ningún producto quede huérfano sin una categoría válida o que se registren ventas de productos inexistentes.

4.2 Diseño de la Capa de Acceso a Datos (DAO)

Para que el código sea modular, se definen los siguientes prototipos de funciones (en un archivo db.h) que envolverán las consultas SQL:

- **int db_init(const char* db_name);**
 - *Propósito:* Abre la conexión con SQLite y crea las tablas si no existen.
- **int db_add_product(Product p);**
 - *Propósito:* Inserta un nuevo registro en la tabla productos.
- **int db_update_stock(int id, int new_stock);**
 - *Propósito:* Actualiza el stock de un producto de forma atómica.
- **int db_register_sale(int id_prod, int qty);**
 - *Propósito:* Registra una venta y descuenta el stock usando una transacción SQL.

- **void db_get_inventory_report();**
 - *Propósito:* Ejecuta un SELECT con JOIN para mostrar el inventario con sus categorías.